Last but not least, the algorithm should be tried and optimized on a diverse set of graphs, to be sure to avoid overfitting it to complete graphs (or, alternatively, when faced with a specific problem, you can tune parameters on small instances of the graphs you expect to see, and once ready apply the tuned version to your real instances).

From what we could see scaling up to K_{10} (figure 17.15), for instance, the configuration we used seems to work well with larger complete graphs.

17.3.2 Force-directed Drawing

In section 16.5 we described a class of graph drawing algorithms called *force-directed drawing* that use a physics-based approach to compute aesthetically-pleasing embeddings of graphs: figure 17.16 reminds us why a nice embedding is important when graphs need to be visualized.

The spring embedder model for drawing undirected graphs was first introduced[14] by Peter Eades in the late 1980s, then refined[15] by Kamada and Kawai, who introduced the idea of optimal edge length, and sequential vertex update (by moving only one vertex at each step).

The algorithm is evolved into a state of minimum energy by using gradient-descent: often, though, by using a deterministic learning technique like gradient descent we are bound to remain stuck in local minima, reaching an equilibrium in for the system, but not the state with the minimum possible energy.